Product Requirements Document (PRD) – RightStaff

AI Team

21 October 2025 (Revised: 01 November 2025)

Table 1: Version and Ownership

Version	1.1 (Revised)
Feature / Increment	AI Candidate Ranking and Job-Scoped Chatbot (MVP)
Owner	RightStaff AI Team
Date	01 November 2025

1 Overview

RightStaff is an intelligent staffing solution that helps staffing agencies match candidates with open roles using an explainable blend of ontology-based filters, semantic retrieval, and re-ranking. This MVP focuses on the AI technology: ingesting candidate resumes into a vector database, expanding skills via a lightweight ontology, generating and storing embeddings, scoring candidates fairly and transparently, and exposing a job-scoped assistant that can answer hiring-manager questions and draft outreach emails. The primary outcome is an ordered list of candidates with scores, reasons, and citations, ready for portal integration. A secondary outcome is a simple UI for internal validation of ranking and chat.

2 Objective and Scope

Objective: deliver a production-grade, explainable candidate ranking engine + grounded chatbot that runs locally (Docker) and is cloud-ready.
Success: fast, fair, low-cost, auditable rankings usable by the Portal team without rework.

In Scope

Highlights: Vector database ingestion, skill normalization, embeddings, semantic retrieval, re-ranking, explanations, grounded chatbot, minimal sandbox UI, observability, security/guardrails.
Detailed List:

- Ingesting candidate resumes from S3, parsing (PDF/DOCX), chunking, and storing embeddings in the vector database (Qdrant for MVP, Pinecone for production).
- Implementing an ontology-based gate for must-have skills and scoring of nice-to-have skills.
- Building semantic retrieval using vector search to compute similarity and surface evidence chunks.

- Deploying a cross-encoder or small language model to judge individual job–candidate pairs and produce pairwise scores and reasons.
- Combining dense, structured and pairwise scores into a single final score; banding candidates; and generating explanations with citations.
- Creating a simple user interface to display ranked lists, resumes and a job-scoped chatbot.
- Ensuring the system adheres to privacy, security and fairness requirements.

Out of Scope (MVP)

- Full applicant tracking workflow (interviews, scheduling, feedback loops) and multi-tenant billing.
- Complex CRM features such as bulk email campaigns, analytics dashboards or integrated payroll.
- Long-term learning from user feedback or adaptive model retraining — these will be considered in future iterations.
- Managing the candidate relational database (handled by Portal team).

3 Team Responsibilities & Integration Points

AI Team Responsibilities (This Team)

- **Vector Database Management**: Own and maintain Qdrant (MVP) / Pinecone (production) for candidate embeddings.
- **Resume Processing Pipeline**:
  - Listen for candidate profile creation/update events from Portal team.
  - Fetch resumes from S3 (uploaded by Portal team).
  - Parse PDF/DOCX and extract text.
  - Chunk text (400 tokens, 50 token overlap).
  - Detect skills in each chunk using ontology.
  - Generate embeddings using `sentence-transformers/all-MiniLM-L6-v2` (MVP) or OpenAI `text-embedding-3-large` (production).
  - Upsert vectors to Qdrant/Pinecone with metadata.
- **Ranking Pipeline**: Implement ontology gating, dense retrieval, structured scoring, pairwise re-ranking, and explanation generation.
- **Chatbot**: Build job-scoped RAG chatbot with WebSocket support.
- **REST API**: Expose endpoints for ranking and candidate explanation retrieval.
- **Mock UI**: Create simple HTML form for testing end-to-end workflow (temporary for MVP validation).

Portal Team Responsibilities (Integration Points)

- **Candidate Relational Database**: Create and maintain PostgreSQL schema for candidate profiles, contacts, preferences, compliance, demographics, and applications.
- **Resume Upload**: Handle candidate resume uploads to S3.
- **Profile Management**: Provide forms for candidates to create/update profiles (Workday-style form-filling + resume upload).
- **Webhook/Event Trigger**: Notify AI team when candidate profile is created or updated, triggering vector database ingestion.

- **Database Access**: Provide read-only access to AI team for candidate and job tables.
- **Frontend Integration**: Consume AI team's REST APIs and WebSocket for ranking and chatbot features.

Data Flow Between Teams

1. **Candidate onboarding**: Portal team captures resume upload + form data → stores in PostgreSQL + S3 → triggers AI team webhook.
2. **AI ingestion**: AI team receives webhook → fetches resume from S3 → processes and stores embeddings in Qdrant.
3. **Profile updates**: Portal team updates candidate data → triggers webhook → AI team re-processes and updates vectors.
4. **Ranking request**: Portal team calls AI endpoint `POST /api/v1/jobs/{job_id}/rank` → AI team queries both PostgreSQL (for gating) and Qdrant (for semantic retrieval) → returns ranked list.
5. **Chatbot**: Portal team connects to AI WebSocket `WS /api/v1/jobs/{job_id}/chat` → AI team provides grounded responses.

4 User and Problem Context

RightStaff serves three primary user groups:

- **Staffing agency agents**: These professionals work for staffing firms and are responsible for sourcing and placing talent. They need a robust ranking mechanism that surfaces qualified candidates quickly, provides transparent reasons for each recommendation, allows them to ask follow-up questions and draft outreach emails, and helps them make confident placement decisions.
- **Volunteer candidates**: Individuals seeking employment opportunities sign up on the platform, create and update their profiles, upload resumes, specify preferences (such as salary requirements, openness to remote work and visa sponsorship), and view job postings. They expect their data to be handled securely and fairly, and they want timely feedback on their fit for roles.
- **Admins**: System administrators and compliance officers manage job postings, candidate and staff profiles, extension points and settings. They ensure that the ontology, skills and compliance data remain up to date, monitor the AI pipeline for fairness and performance, and oversee consent and volunteer compliance flags.

To meet compliance goals and avoid bias, the AI engine must ignore protected attributes (such as disability, ethnicity or veteran status) in the ranking algorithm and store such data separately for reporting purposes. The system must also handle large resumes and job descriptions. PII is protected and access is audited.

5 Requirements and User Stories

Functional Requirements

- **Vector database ingestion** (AI team): receive webhook from Portal team on candidate creation/update; fetch resume from S3; parse, chunk, embed, and upsert to Qdrant (MVP) / Pinecone (production).

- **Ontology gate**: expand must-haves via parents/synonyms; candidates must satisfy all expanded must-haves to proceed.
- **Dense retrieval**: embed job & candidate artifacts; perform ANN search to compute similarity and surface evidence chunks.
- **Structured scoring**: compute nice-to-have coverage, years, recency, domain match, remote eligibility.
- **Pairwise judgement**: small cross-encoder/LLM scores job–candidate pair and returns reasons + risk flags (JSON).
- **Blending & banding**: combine signals into final score; band by confidence; generate explanations with citations.
- **Job-scoped chatbot**: answer questions grounded in stored data and evidence; draft outreach emails.
- **API exposure**: REST endpoints for ranking; WebSocket for chatbot.
- **Mock UI** (AI team, MVP only): simple HTML form for candidate profile submission to test end-to-end workflow.

Non-Functional Requirements

- **Performance**: rank up to 1,000 candidates/job in < 5s p95; chatbot response < 2.5s.
- **Reliability**: 99.5% API success; idempotent run IDs; structured logs.
- **Cost**: inference $<$ \$0.03 per rank run (default path). MVP uses free local embedding model to minimize costs.
- **Explainability**: every recommendation cites at least one evidence chunk.
- **Fairness**: exclude protected attributes; diminishing-returns curves for experience; normalization per job; feature-attribution logging.

User Stories (with Acceptance Criteria (AC))

- **US1: Ingest Candidate into Vector Database**
  As an AI system, I want to receive a webhook when a candidate profile is created/updated so that I can process their resume and store embeddings.
  Acceptance Criteria:
  - System receives webhook with candidate ID from Portal team.
  - System fetches resume from S3 using provided URL.
  - Resume is parsed (PDF/DOCX), chunked (400 tokens, 50 overlap), and skills detected.
  - Three vector types generated: profile, skills, and chunks (multiple).
  - Vectors upserted to Qdrant with metadata (candidate_id, kind, skills, location, etc.).
  - If candidate updates profile, existing vectors are updated/replaced.
- **US2: Generate Ranked List**
  As a staffing agent, I want to submit a job ID and receive an ordered list of qualified candidates with explanations so that I can review the best fits quickly.
  Acceptance Criteria:
  - Candidates lacking expanded must-have skills are excluded (ontology gate using PostgreSQL).
  - For each remaining candidate, the system queries Qdrant for semantic similarity and produces a final score derived from dense, structured and pairwise signals.
  - The response includes a short summary, 3–5 bullet reasons and at least one evidence snippet ID for each candidate.

  - Candidates are grouped into high, medium and low bands based on confidence.
- **US3: Chat about a Candidate**
  As a staffing agent, I want to ask the system follow-up questions about a candidate's experience or skills and draft an email to them so that I can make an informed outreach.
  Acceptance Criteria:
  - The chatbot answers within the context of the selected job and candidate only.
  - Responses cite stored data from PostgreSQL and evidence snippets from Qdrant; do not fabricate unknown facts.
  - The draft email includes candidate name, job title and personalised bullet points drawn from the explanations.

6 Design and User Flow

The MVP is a deterministic pipeline with two phases: ingest/normalization (A), ontology gating (B) in phase 1, semantic retrieval (C), and re-ranking/explanation (D), in phase 2. Figure 1 shows the flow. Portal team manages candidate onboarding; AI team processes vectors; agents request rankings.

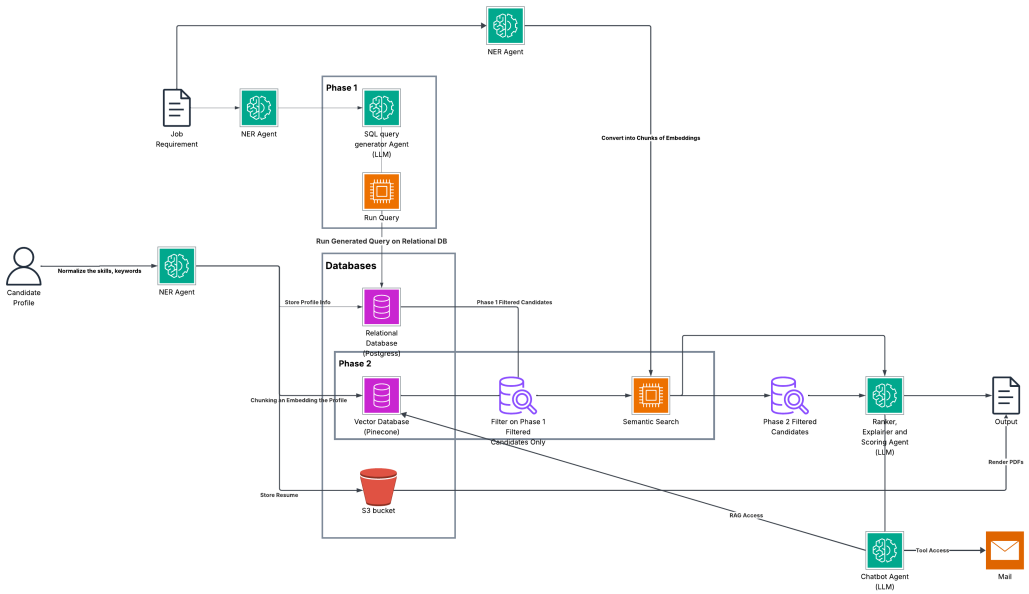

Figure 1: High-level AI workflow: Portal team uploads → AI team chunks/embeds → ontology gate → semantic retrieval (Qdrant/Pinecone) → re-rank → explanations.

Relational Data Model (Managed by Portal Team, Read by AI Team)

The relational database (3NF) separates concerns to eliminate redundancy. Portal team maintains: candidates, contacts, preferences (including visa sponsorship), demographics, compliance, resumes (S3 links), skills & synonyms, interests, jobs, applications, and audit events. AI team has read-only access for ranking pipeline. Figure 2 is the visual description of the schema. Table 2 contains the summary of each table in the schema.

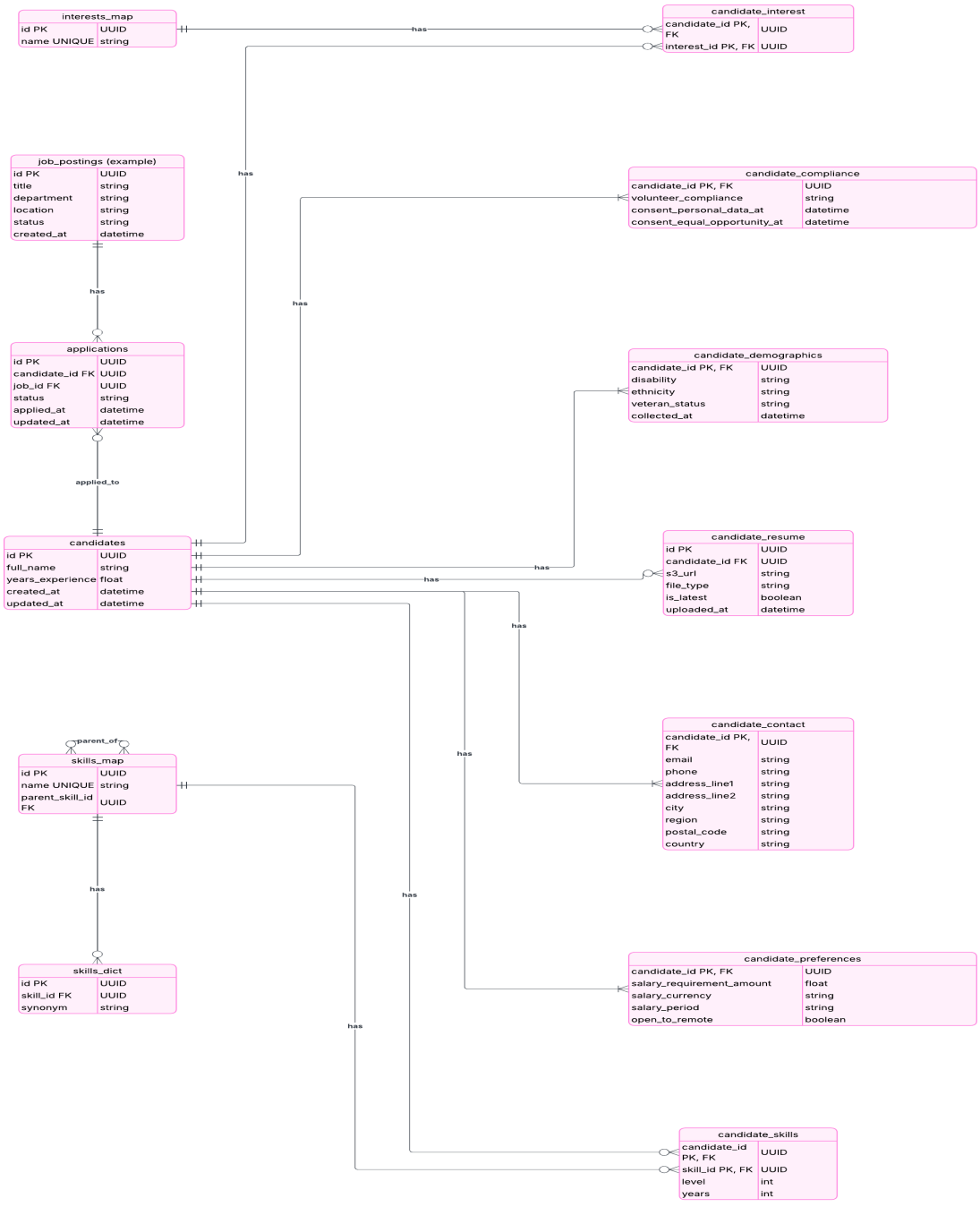

Figure 2: Relational schema (3NF): candidates, contacts, preferences, demographics, compliance, resumes, skills, interests, jobs, applications. (Managed by Portal Team)

Table 2: Key Tables (summary) – Portal Team Responsibility

Table	Key Attributes (PK underlined)
candidates	id (UUID), full_name, years_experience, created_at, updated_at
candidate_contact	candidate_id (UUID), email, phone, address_line1, address_line2, city, region, postal_code, country
candidate_preferences	candidate_id (UUID), salary_requirement_amount, salary_currency, salary_period, open_to_remote, visa_sponsorship_required_at_hire (bool)
candidate_resume	id (UUID), candidate_id (FK), s3_url, file_type, is_latest (bool), uploaded_at
candidate_demographics	candidate_id (UUID), disability, ethnicity, veteran_status, collected_at
candidate_compliance	candidate_id (UUID), volunteer_compliance, consent_personal_data_at, consent_equal_opportunity_at
skills_map	id (UUID), name (unique), parent_skill_id (FK)
skills_dict	id (UUID), skill_id (FK), synonym
candidate_skills	candidate_id (UUID), skill_id (UUID), level (int), years (int)
interests_map	id (UUID), name (unique)
candidate_interest	candidate_id (UUID), interest_id (UUID)
job_postings	id (UUID), title, department, location, status, created_at
applications	id (UUID), candidate_id (FK), job_id (FK), status, applied_at, updated_at

Vector Database (Qdrant for MVP, Pinecone for Production)

MVP Configuration (Qdrant):

- **Embedding model:** `sentence-transformers/all-MiniLM-L6-v2` (384-dim, free, local)
- **Distance metric:** Cosine similarity
- **Collection name:** `candidates_v1`
- **Deployment:** Self-hosted via Docker

Production Configuration (Pinecone):

- **Embedding model:** OpenAI `text-embedding-3-large` (3072-dim)
- **Distance metric:** Cosine similarity
- **Index name:** `candidates`
- **Deployment:** Pinecone managed cloud

Vector IDs & items (consistent across both):

- `cand_{ID}_profile` – candidate profile vector.
- `cand_{ID}_skills` – candidate skills vector (weighted skills text).
- `cand_{ID}_chunk_{k}` – resume chunk vectors (400 tokens, stride 50), with page and index for citations.

Metadata schema (attached to every vector):

- `candidate_id` (UUID), `kind` (`profile|skills|chunk`)
- `city`, `region`, `country`, `seniority`
- `skills` (array of canonical names) *and* `skill_ids` (UUIDs from Postgres)
- chunk-only: `chunk_index` (int), `page` (int), `text_preview` (short, PII-redacted), `source_uri_s3`
- housekeeping: `updated_at` (ISO8601), `embedding_model_id` (e.g., `st_all-MiniLM-L6-v2` for MVP or `oai_text-embedding-3-large` for production)

Upsert example candidate profile (C1):

```
("cand_C1_profile", cand_C1_profile_vec,
 {"candidate_id":"C1","kind":"profile","city":"Austin",
  "seniority":"mid","embedding_model_id":"st_all-MiniLM-L6-v2"}),

("cand_C1_skills", cand_C1_skills_vec,
 {"candidate_id":"C1","kind":"skills",
```

```
  "skills":["Python","NLP","Transformers","FastAPI","Postgres"]}),

("cand_C1_chunk_07", cand_C1_chunk07_vec,
 {"candidate_id":"C1","kind":"chunk","chunk_index":7,"page":12,
  "text_preview":"Deployed Transformer-based NER to production FastAPI...",
  "skills_detected":["NLP","NER","Transformers","FastAPI"],
  "source_uri_s3":"s3://bucket/raw/C1/resume.pdf"})
```

Query policy (per job):

1. Compute `job_profile_vec` from job title+summary; `job_skills_vec` from expanded skills.
2. Filter to SQL-gated set S' (city/remote, work auth, salary) from PostgreSQL.
3. Query vector database:
   - *Profiles:* `filter = {kind:"profile", candidate_id:{`$\in S'$`}}`
   - *Skills:* `filter = {kind:"skills", candidate_id:{`$\in S'$`}}`
   - *Chunks (evidence):* `filter = {kind:"chunk", candidate_id:{`$\in S'$`}}`
4. Optional filters available: `city`, `remote_ok`, `seniority`, domain tags.

Dense score: for each candidate,

$$s_{\text{dense}} = 0.5\, s_{\text{profile}} + 0.3\, s_{\text{skills}} + 0.2\, s_{\text{chunk}}.$$

Keep the top chunk ID/snippet per candidate for explanations.

PII: redact emails/phones from `text_preview`; retain full text in S3 for regulated access.

7 Technical Details

Scoring Mechanism

A candidate must first satisfy all expanded must-haves (SQL query against `candidate_skills` and `skills_map`). For each survivor:

1. **Dense similarity** s_{dense}: cosine similarities between job/candidate profiles, skills, and best evidence chunk (weighted as above). Query vector database (Qdrant/Pinecone). Normalize to percentiles within S'.
2. **Structured features** $s_{\text{structured}}$: objective facts from PostgreSQL (nice-to-have coverage, years with saturation, recency, domain match, remote):

$$\begin{aligned} s_{\text{structured}} = {} & 0.35 \cdot \text{must\_have\_coverage} + 0.20 \cdot \text{nice\_to\_have\_score} + \\ & 0.20 \cdot \text{years} + 0.15 \cdot \text{recency} + 0.10 \cdot \text{domain\_match}. \end{aligned}$$

3. **Pairwise judgement** s_{cross}: a cross-encoder (e.g., `bge-reranker-base`) or small LLM scores the pair (job text + candidate resume chunks from vector DB) and returns reasons + risk flags.

Final blend:

$$\text{final\_score} = 0.40\, s_{\text{dense}} + 0.35\, s_{\text{cross}} + 0.25\, s_{\text{structured}}.$$

Band candidates (High/Medium/Low) by percentile; explanations must cite at least one evidence chunk.

Model & Framework Selection

- **NER/Skill extraction:** spaCy + PhraseMatcher; fallback Llama 3.1 8B/Mistral 7B on low-confidence.
- **Embeddings (MVP):** `sentence-transformers/all-MiniLM-L6-v2` (384-dim, free, local).
- **Embeddings (Production):** OpenAI `text-embedding-3-large` (3072-dim).
- **Vector store (MVP):** Qdrant (self-hosted via Docker).
- **Vector store (Production):** Pinecone (managed cloud).
- **Cross-encoder:** `bge-reranker-base`; alt: Cohere Rerank-3.
- **Explanation writer & Chatbot:** GPT-4o-mini by default; alt: Llama 3.1 8B self-hosted.
- **Orchestration:** LangGraph (LangChain) for a deterministic DAG; LlamaIndex for parsing/chunking helper utilities.
- **API Framework:** FastAPI with WebSocket support.

8 API Contract

REST Endpoints (Exposed by AI Team)

- **POST /api/v1/candidates/ingest** (MVP testing only)
  Purpose: Mock endpoint for testing candidate ingestion via simple HTML form.
  Request: Multipart form with resume file + profile JSON.
  Response: {`candidate_id, status, vectors_created`}
- **POST /api/v1/jobs/{job_id}/rank**
  Purpose: Generate ranked candidate list for a job.
  Request:

```
{
  "run_id": "uuid",
  "filters": {
    "city": ["Austin", "Remote"],
    "min_years_experience": 3,
    "max_salary_requirement": 120000
  }
}
```

  Response:

```
{
  "run_id": "uuid",
  "job_id": "uuid",
  "ranked_candidates": [
    {
      "candidate_id": "uuid",
      "final_score": 0.87,
      "band": "high",
      "summary": "Strong ML engineer with 5+ years in NLP",
      "reasons": ["Exact match on required skills...", ...],
      "evidence_snippets": [{...}],
      "risk_flags": []
    }
```

```
  ],
  "metadata": {
    "total_candidates_considered": 247,
    "gated_out": 189,
    "ranked": 58,
    "processing_time_ms": 3421
  }
}
```

- **GET /api/v1/candidates/{candidate_id}/explanation**
  Purpose: Retrieve stored explanation for a candidate's ranking.
  Response: {`candidate_id, summary, reasons, evidence_snippets`}
- **GET /api/v1/health**
  Purpose: Health check for service monitoring.

WebSocket Endpoint

- **WS /api/v1/jobs/{job_id}/chat**
  Purpose: Job-scoped chatbot for candidate Q&A and email drafting.
  Protocol: Send JSON messages; receive streaming responses.
  Example:

```
// Client sends
{"message": "Does candidate C have AWS certification?",
 "candidate_id": "uuid"}

// Server responds
{"response": "Yes, candidate has AWS Solutions Architect...",
 "citations": ["cand_C_chunk_15"],
 "confidence": 0.92}
```

9 Testing and Validation

Unit, integration, and UAT tests cover parsing, ontology expansion, retrieval quality, re-ranking, explanations (with citations), chatbot grounding, fairness counterfactuals, and load/perf (Locust). Sign-off requires meeting latency, accuracy, and explainability SLOs.

Test Coverage

- **Unit tests**: Resume parsing, chunking, skill detection, embedding generation, vector upsert.
- **Integration tests**: End-to-end ranking pipeline, PostgreSQL + Qdrant queries, API endpoints.
- **Chatbot tests**: RAG grounding, citation accuracy, hallucination prevention.
- **Performance tests**: Latency targets (5s for ranking, 2.5s for chatbot), concurrent requests.
- **Fairness tests**: Counterfactual analysis, protected attribute exclusion verification.

10 Compliance and Security

Encrypt PII at rest/in transit, enforce consent and data-minimization, exclude protected attributes from features, retain immutable audit logs, use RBAC and rate limits, and honor deletion requests per policy.

Key Security Measures

- **PII Protection**: Redact sensitive info in logs and vector metadata.
- **Access Control**: Read-only PostgreSQL access for AI team; no write permissions.
- **Audit Trails**: Log all ranking runs, chatbot queries, and vector updates.
- **Bias Mitigation**: Exclude `candidate_demographics` table from all ranking logic.
- **Data Deletion**: Support candidate right-to-be-forgotten (delete vectors + relational records).

11 Release Plan (2 Weeks)

Table 3: Milestones

Stage	Owner	Deliverable	Target
Repo, schema, Docker setup	AI Team	Docker Compose (Postgres + Qdrant + MinIO), Alembic	Day 1
Mock UI & ingestion	AI Team	HTML form for candidate submission, S3 upload, webhook trigger	Day 3
Resume parsing & chunking	AI Team	PDF/DOCX extraction, text chunking (400 tokens), skill detection	Day 4
Embedding pipeline	AI Team	sentence-transformers integration, Qdrant upserts with metadata	Day 5
Ontology gate	AI Team	Skills tables + expansion logic (SQL queries)	Day 6
Dense retrieval	AI Team	Qdrant queries (profile/skills/chunks), similarity scoring	Day 7
Structured scoring	AI Team	Nice-to-have, years, recency, domain logic	Day 8
Re-rank & explain	AI Team	Cross-encoder, score blending, explanation generator	Day 10
REST API	AI Team	/rank endpoint with full pipeline, /explanation endpoint	Day 11
Chatbot & WebSocket	AI Team	Job-scoped RAG, email drafting, WS implementation	Day 12
Testing/observability	AI Team	Unit/integration tests, bias checks, logging	Day 13
Internal demo & handoff	AI Team	Working MVP, API docs, integration guide	Day 14

12 Outcomes and Learnings

We will track acceptance rate (Top-5), false negatives, explanation usefulness, p50/p95 latency per phase, Qdrant QPS (MVP) / Pinecone RCU (production), cost per run, parse success rates, synonym hit-rate, and fairness drift. Findings will inform the next iteration.

Success Metrics

- **Ranking Quality**: Agent thumbs-up rate on top-5 candidates 80%.
- **Performance**: p95 latency for ranking < 5s; chatbot response < 2.5s.

- **Cost**: MVP using local embeddings: $0 per run; Production target: < $0.03 per run.
- **Explainability**: 100% of ranked candidates have at least one cited evidence chunk.
- **Fairness**: Zero correlation between rankings and protected attributes in audit logs.
- **Resume Processing**: 95% successful parse rate for PDF/DOCX files.
- **Vector Sync**: < 30s end-to-end latency from webhook trigger to Qdrant upsert completion.

13 Migration Path: MVP to Production

MVP Stack (Development & Testing)

- **Vector DB:** Qdrant (Docker, localhost)
- **Embeddings:** sentence-transformers/all-MiniLM-L6-v2 (384-dim, CPU)
- **Storage:** MinIO (S3-compatible, Docker)
- **Cost:** $0 for vector operations
- **Latency:** 20ms per embedding, 50ms per vector search

Production Stack (Deployment)

- **Vector DB:** Pinecone (managed cloud, auto-scaling)
- **Embeddings:** OpenAI text-embedding-3-large (3072-dim, API)
- **Storage:** AWS S3
- **Cost:** $0.02 per ranking run (estimated)
- **Latency:** 100ms per embedding batch, 30ms per vector search

Migration Steps

1. **Configuration switch**: Update environment variables to point to Pinecone API and OpenAI API.
2. **Re-embedding**: Batch process all existing candidates with new 3072-dim embeddings.
3. **Index creation**: Create Pinecone index with cosine metric, migrate metadata schema.
4. **Testing**: Verify ranking quality matches or exceeds MVP baseline.
5. **Cutover**: Update webhook to trigger Pinecone upserts; deprecate Qdrant.

Downtime: Zero (dual-write during transition, blue-green deployment).

14 Appendix A: Webhook Payload Specification

Webhook: Candidate Profile Created/Updated

Endpoint: `POST /api/v1/webhooks/candidate-updated` (AI team exposes this)

Request from Portal Team:

```
{
  "event_type": "profile_created" | "profile_updated" | "resume_uploaded",
  "candidate_id": "uuid",
  "timestamp": "2025-11-01T10:30:00Z",
  "s3_resume_url": "s3://rightstaff-resumes/candidate-uuid/resume.pdf",
  "profile_snapshot": {
    "full_name": "Jane Doe",
    "city": "Austin",
```

```
    "region": "TX",
    "years_experience": 5,
    "skills": ["Python", "Machine Learning", "FastAPI"]
  }
}
```

Response to Portal Team:

```
{
  "status": "accepted",
  "candidate_id": "uuid",
  "processing_job_id": "async-task-uuid",
  "estimated_completion_seconds": 30
}
```

AI Team Processing:

1. Acknowledge webhook immediately (return 202 Accepted).
2. Queue async job: fetch resume from S3 → parse → chunk → embed → upsert to Qdrant.
3. Log processing result (success/failure) for monitoring.

15 Appendix B: Mock UI Specification (MVP Only)

Purpose

Provide a simple HTML form for AI team to test end-to-end workflow without waiting for Portal team's frontend.

Features

- **Resume Upload**: File input (PDF/DOCX), max 5MB.
- **Profile Form**: Text fields for name, email, phone, city, years experience.
- **Skills Input**: Multi-select dropdown populated from `skills_map` table.
- **Preferences**: Checkboxes for remote work, visa sponsorship; salary input.
- **Submit Button**: POST to `/api/v1/candidates/ingest` endpoint.
- **Result Display**: Show candidate ID, vector count, processing time.

Tech Stack

- **Frontend**: Single HTML file with vanilla JavaScript (no build step).
- **Styling**: Minimal CSS (responsive, accessible).
- **Serving**: FastAPI serves static file at `/mock-ui`.

Not Included (Out of Scope)

- Authentication/authorization (assumes local testing).
- Advanced form validation (basic HTML5 validation only).
- Job creation UI (use direct API calls or SQL inserts).
- Production-ready error handling.

16 Appendix C: Technology Stack Summary

Table 4: Complete Technology Stack

Component	MVP	Production
Backend Framework	FastAPI 0.104+	FastAPI 0.104+
Vector Database	Qdrant (Docker)	Pinecone (Cloud)
Relational DB	PostgreSQL 15+ (Docker)	AWS RDS PostgreSQL
Object Storage	MinIO (Docker)	AWS S3
Embeddings	sentence-transformers (local)	OpenAI API
LLM (Chatbot)	GPT-4o-mini (API)	GPT-4o-mini (API)
Cross-Encoder	bge-reranker-base (local)	bge-reranker-base (local)
NLP	spaCy 3.7 (local)	spaCy 3.7 (local)
Orchestration	LangChain + LangGraph	LangChain + LangGraph
Migrations	Alembic	Alembic
Testing	pytest, locust	pytest, locust
Logging	structlog (JSON)	CloudWatch / Datadog
Deployment	Docker Compose	Kubernetes / ECS
CI/CD	GitHub Actions	GitHub Actions

17 Appendix D: Glossary

- **ANN**: Approximate Nearest Neighbor search for fast vector similarity.
- **Chunking**: Splitting long documents into smaller segments (400 tokens) for embedding.
- **Cross-Encoder**: Model that scores relevance of query-document pairs (used in re-ranking).
- **Dense Retrieval**: Semantic search using vector embeddings.
- **Embedding**: Numerical vector representation of text (384-dim or 3072-dim).
- **Ontology**: Hierarchical structure of skills with parent-child and synonym relationships.
- **RAG**: Retrieval-Augmented Generation (chatbot pattern: retrieve context + generate answer).
- **3NF**: Third Normal Form (database design principle to eliminate redundancy).
- **Webhook**: HTTP callback triggered by Portal team when candidate profile changes.

18 Appendix E: Risk Register

Table 5: Key Risks and Mitigations

Risk	Probability	Impact	Mitigation
Resume parsing failures	Medium	High	Fallback to text extraction only; log failures for manual review; support 95%+ of common formats.
Embedding quality insufficient	Low	High	Start with free local model; benchmark against OpenAI; switch if accuracy $<$ 80%.
Latency exceeds SLA	Medium	Medium	Profile hot paths; cache embeddings; optimize SQL queries; use async processing.
Portal team delays DB schema	High	Critical	Use mock schema from teammate's SQL files; generate test data; maintain decoupling via API contract.
Qdrant $\rightarrow$ Pinecone migration issues	Low	Medium	Maintain identical metadata schemas; test with subset; automate re-embedding pipeline.
Bias in rankings	Low	Critical	Exclude demographics table; log feature attributions; run fairness audits; peer review scoring logic.
API integration friction	Medium	High	Provide OpenAPI docs; create integration examples; maintain backward compatibility; version endpoints.

19 Appendix F: Future Enhancements (Post-MVP)

Phase 2 Features (3-6 Months)

- **Adaptive Learning**: Collect agent feedback (thumbs up/down) to fine-tune ranking weights.
- **Multi-Job Matching**: Recommend top 3 jobs for each candidate (reverse ranking).
- **Interview Scheduling**: Integrate with calendar APIs (Calendly, Google Calendar).
- **Advanced Analytics**: Dashboard for ranking quality metrics, conversion funnels.
- **Bulk Operations**: Batch re-rank all candidates when job description changes.

Phase 3 Features (6-12 Months)

- **Fine-Tuned Embeddings**: Train domain-specific embedding model on recruitment data.
- **Multi-Modal Support**: Extract skills from LinkedIn profiles, GitHub repos.
- **Real-Time Ranking**: Stream ranking updates as new candidates apply.
- **Multi-Tenancy**: Support multiple staffing agencies with isolated data.
- **Explainable AI UI**: Interactive visualizations of ranking decisions.

Document History

Version	Date	Changes
1.0	21 Oct 2025	Initial draft
1.1	01 Nov 2025	Clarified team responsibilities; switched to Qdrant (MVP) + Pinecone (prod); added webhook spec; updated chunking to 400 tokens; added migration path; added mock UI requirements